
JOURNAL D'ASSISTANCE IA

Outils, tâches, analyse critique & limites observées

Équipe SmartCampus

Viktor GOUSSOT

Nicolas CARMINATI

Louis PEZE

Alexis OSORIO

Journal d'assistance par IA — SmartCampus

Le sujet autorise l'usage d'outils d'IA générative et **évalue le recul critique**, pas l'abstinence. Ce journal documente honnêtement comment l'IA a été utilisée. **À personnaliser** : adaptez les exemples à votre vécu réel et complétez avec vos propres échanges.

1. Outils utilisés

Outil	Usage principal
Assistant IA (Claude / ChatGPT)	Aide à la conception (modèle de données), génération de squelettes de code, explication de mécanismes (CSRF, PDO), débogage
Documentation officielle	PHP (PDO, password_hash), MDN (fetch, ES modules), Bootstrap, FPDF

2. Tâches assistées par IA

- Conception du **modèle entité-association** et discussion des cardinalités (héritage `users` → profils).
- Mise en place du **noyau backend** (routeur REST, réponses JSON, validation).
- Implémentation des **règles métier** (détection de conflits d'emploi du temps en SQL).
- Génération de **composants frontend répétitifs** (formulaires modaux, tableaux).
- Débogage** (ex. génération PDF cassée par une alerte PHP).

3. Réponses utiles

- La structure d'un **mini-MVC sans framework** proposée par l'IA était claire et facile à expliquer ; nous l'avons adoptée.
- La requête SQL de **détection de chevauchement horaire** (`start_time < ? AND end_time > ?`) était correcte et concise.
- Les bonnes pratiques de **sécurité** (hash bcrypt, requêtes préparées, cookie HttpOnly, jeton CSRF) ont été correctement rappelées.

4. Réponses incorrectes / à corriger (recul critique)

C'est la partie la plus importante pour la note. Exemples réels rencontrés :

- Génération PDF cassée** : la première version utilisait FPDF sans le dossier de polices (`font/`), ce qui produisait un PDF corrompu (une alerte PHP s'insérait avant le binaire). **Correction** : ajout du dossier `font/` + désactivation de l'affichage des erreurs sur l'endpoint binaire.
- Tendance à proposer un framework** (Laravel) : écarté car le sujet déconseille les générateurs produisant l'architecture ; nous voulions un code **maîtrisé et explicable**.

- **JWT proposé pour l'authentification** : remplacé par des **sessions PHP**, plus simples et suffisantes pour une application servie en même origine.
- **Validation uniquement côté client** suggérée initialement : nous avons re-validé **systématiquement côté serveur** (le client peut être contourné).
- **Identifiants de base de données en clair** dans le code : déplacés dans un `config.local.php` non versionné.

5. Limites observées

- L'IA peut produire du code **plausible mais incomplet** (dépendances manquantes comme les polices FPDF) : il faut **tester réellement**.
- Elle ne connaît pas notre **contexte précis** (mot de passe MySQL, version de PHP) : l'adaptation reste manuelle.
- Elle peut **sur-dimensionner** (proposer des outils non demandés) : tri nécessaire au regard des contraintes pédagogiques.

6. Notre démarche de validation

Pour chaque portion générée : (1) **lire et comprendre** le code, (2) **l'adapter** à notre modèle/nommage, (3) **le tester** (API testée à la main + scénarios de règles métier), (4) **savoir l'expliquer** en soutenance. Toute fonctionnalité présentée peut être justifiée techniquement et fonctionnellement par l'équipe.